CP12 — Proposta de upgrade do projeto N1

Internet das Coisas | Aula 12 — Hardware, firmware-base e contratos

Prof. Edson Vaz Lopes

Identificação

Equipe (nº e nome):	**Projeto (N1 → N3):**
Equipe HorangoTango	Garagem inteligente: monitoramento de vagas → produto **HorangoTango Park**
Integrantes presentes:	**Data / Versão:**
Gustavo Vinicius Taques, Luis Fernando Pereira, João Pedro Angelico, Vynicyus Candido	27/10/2026 – v1

Orientações

Este checkpoint é a **proposta de upgrade** do projeto que a equipe defendeu na N1 para o produto da N3: cada seção confere o plano da equipe contra o piso **R1–R18** definido em `requisitos_projeto_N3.pdf` (documento à parte, **leitura obrigatória** antes de preencher este checkpoint).

Da Aula 12 até a N2 (03/11), a Unidade 2 é **100% desenho**: não há bancada, não há circuito montado, não há firmware rodando no ESP32. O que se produz é documentação, cronograma, fluxogramas, diagrama de blocos e repositório GitHub organizado. A **Unidade 3** (10/11, 17/11, 24/11) é que **põe em prática** o que foi desenhado aqui, até a defesa do produto funcionando na N3 (01/12). Código mostrado neste checkpoint é **aproximado**: pode e deve mudar quando for implementado de fato no ESP32.

Entrega em duas etapas, hoje – duas coisas em cada uma: o PDF no Teams e o repositório no GitHub.

- **Até 22h30 de hoje (20/10):**
  - PDF com as seções 1 a 4 preenchidas – ponto de partida, upgrade por camada, prova de que a equipe leu o piso R1–R18 e já mapeou cada requisito para o próprio projeto, e a arquitetura-alvo esboçada (diagrama de blocos, fluxo do dado e fluxo do comando). Nenhuma dessas quatro seções depende de teoria técnica nova – usam o que já foi visto na Aula 11 e a leitura autônoma do piso.

  - Repositório GitHub criado (se ainda não existir) ou atualizado, com um `README.md` simples: nome do projeto, problema e usuário, uma frase sobre o estado atual (herdado da N1) e para onde o upgrade está indo. Não precisa de instruções de instalação ainda – isso só é exigido no fim (R11, válido para a N3).

- **Até o final da Aula 13 (27/10/2026, 22h30):** PDF completo, com as seções 5 a 9 (hardware calculado, firmware em fluxograma, contrato congelado v1, tecnologias, cronograma/riscos/divisão), e repositório atualizado de novo (seção 9). Essas seções dependem da teoria de hardware seguro, firmware-base e contratos, dada na própria Aula 13 – a oficina daquela noite é o tempo de aplicar o conteúdo e fechar o documento.

Todos os integrantes devem revisar o documento inteiro antes do envio final: o conteúdo é objeto da prova individual da N2 (03/11). Este checkpoint é a primeira (e única) entrega do **T2 – desenho e documentação do projeto** (grupo, 40% da N2); a entrega completa, na Aula 13, já congela o contrato como v1 (não há um CP13 separado). A N2 (03/11) = T2 (40%) + P2, prova individual de 6 questões (60%) – o conteúdo deste checkpoint é também o que cai na prova.

A partir de hoje, pensem o projeto como **produto**, não como exercício acadêmico (ver subseção *Como apresentar* em `requisitos_projeto_N3.pdf`): proposta de valor clara, cuidado visual e praticidade de uso. Isso não tem campo de pontuação próprio neste checkpoint; é a lente que vale desde já para a interface e o acabamento do hardware.

1. Ponto de partida: o projeto da N1

Parte da entrega parcial de hoje (22h30).

Problema e usuário	Em garagens e estacionamentos, o motorista perde tempo circulando à procura de vaga, e o operador não sabe em tempo real quantas vagas estão livres nem há quanto tempo cada uma está ocupada. **Usuários:** (1) o **motorista**, que quer saber antes de entrar onde há vaga e ser guiado até ela; (2) o **operador/administrador** do estacionamento, que monitora a ocupação, configura o sistema e reserva vagas.
O que funcionou na N1	Sensor ultrassônico HC-SR04 por vaga com histerese (25/35 cm), confirmação por 3 leituras e detecção de falha do sensor (`erro` após 5 leituras inválidas); ESP32 publicando o estado da vaga em MQTT com *retained*; Last Will (`online`/`offline`) por nó; ESP32 atuador com LEDs verde/vermelho assinando o broker (sensor e atuador sem ligação direta); app web de vagas livres via MQTT/WebSocket; broker Mosquitto local documentado; credenciais Wi-Fi fora do código (`secrets.h` no `.gitignore`).
Limitações conhecidas	Sem backend, sem banco e sem histórico; a interface fala direto com o broker; payload em texto puro (`livre`/`ocupada`), sem `deviceId`, timestamp nem unidade; nenhum comando do usuário para o dispositivo (só leitura); broker sem autenticação (`allow_anonymous true`); limiares fixos no código (não configuráveis remotamente); os LEDs dependiam do broker, então sem rede a vaga ficava sem indicação; o sistema não sabe **qual** carro está na vaga.

Proposta de valor (por que alguém usaria/compraria este produto)	O motorista abre o app no celular, vê na hora quais vagas estão livres e, na garagem, é guiado pelos LEDs: verde = livre, vermelho = ocupada, azul = reservada. Clientes cadastrados (carro com tag RFID) são identificados ao estacionar e podem ter vaga reservada. O operador ganha ocupação em tempo real, histórico de uso (taxa de ocupação, tempo médio de permanência), alerta no celular quando um sensor falha e reservas com confirmação do hardware — sem precisar de alguém circulando para conferir as vagas.

Pedido de troca de projeto: ☐ Sim ☒ Não

Se sim, justificar por que o projeto da N1 não comporta o upgrade e qual problema/usuário a equipe propõe no lugar. Só é válido com aprovação do professor.

Histórico da N1: a proposta inicial, de 26/08 (`docs/IOT_24_08.pdf`), era uma balança de encomendas com sensor de peso. Em 14/09 a equipe a substituiu pela garagem inteligente, por causa dos recursos de hardware disponíveis e do caminho de crescimento até a N3 com esse hardware. A garagem é o projeto que funcionou e foi apresentado na N1. Decisão registrada em `docs/adr/0002-troca-sensor-de-peso-por-garagem.md`.

2. Upgrade por camada

Parte da entrega parcial de hoje (22h30).

Para cada camada, descrever o estado na N1 e o que muda na N3.

Camada	Como era na N1	Como será na N3
Hardware	2 ESP32 (sensor e atuador), HC-SR04 por vaga, LEDs verde/vermelho por vaga.	1 ESP32 DevKit (WROOM-32) operando 2 vagas: por vaga, 1 HC-SR04 (presença do carro), 1 leitor RFID RC522 (identificação do carro com tag) e 1 LED RGB (verde/vermelho/azul). Divisor de tensão no Echo, alimentação USB 5 V dimensionada, maquete com acabamento.
Firmware	Dois sketches; leitura com histerese e confirmação; publicação só na mudança; reconexão MQTT; LED só via broker.	Firmware único e não bloqueante: ultrassônico (1 s), RFID (100 ms), LED com **regra local** (funciona sem rede), estados *automático / reservado (manual) / seguro*, execução de comandos com confirmação, configuração persistente em NVS (limiares e tempo de reserva), `deviceId` pelo MAC, horário por NTP (ou "hora não sincronizada"), logs no Serial, OTA.

Camada	Como era na N1	Como será na N3
Comunicação (MQTT)	Tópicos `garagem/vaga/N` e `garagem/no/<id>/status`, payload texto, sem contrato escrito.	Contrato versionado (`CONTRATOS.md`, v0 → v1 congelado em 27/10): tópicos de telemetria, estado, evento RFID, comando, confirmação e status com Last Will; payload JSON com `v`, `deviceId`, `ts` e unidades; QoS e *retain* justificados por tópico.
Backend	(não existia)	Serviço FastAPI que assina os tópicos do dispositivo, valida cada mensagem contra o contrato (descarta e registra as inválidas), grava no PostgreSQL, executa as regras remotas, envia notificações e expõe a API REST autenticada usada pela interface. É o **único** que publica comandos.
Banco de dados	(não existia)	PostgreSQL com histórico de medições e de estados das vagas, leituras RFID, registro de comandos (commandId, origem, ação, envio, resultado, confirmação), alertas e usuários. Esquema versionado (migrações).
Interface	App web único que assinava o broker direto e mostrava vagas livres.	(1) **Painel do operador** (web, com login): estado atual das vagas, histórico e gráficos, online/offline e último contato do dispositivo, alertas ativos, tags lidas, envio de reserva e alteração de configuração. (2) **App do motorista** (web responsivo no celular): vagas livres em tempo real e destaque da vaga do motorista com tag. Ambos falam só com a API do backend.
Decisão/regra	Só local: histerese e confirmação no ESP32.	**Local (firmware):** histerese, confirmação, estado seguro em falha de sensor e LED seguindo o sensor mesmo sem rede. **Remota (backend):** reserva expira se o carro não chegar no tempo configurado → backend envia comando `cancelar_reserva`; sensor em `erro` ou dispositivo `offline` por mais de 1 min → alerta e notificação no Telegram.

3. Conferência contra o piso R1–R18

Parte da entrega parcial de hoje (22h30).

Para cada requisito do piso (ver `requisitos_projeto_N3.pdf`), uma frase sobre como o projeto pretende atendê-lo. Quem ainda não sabe, escreve a pendência – não deixar em branco. Vários

requisitos de R12 a R18 já foram implementados por algumas equipes na N1: nesse caso, descrever o que já existe e o que falta evoluir.

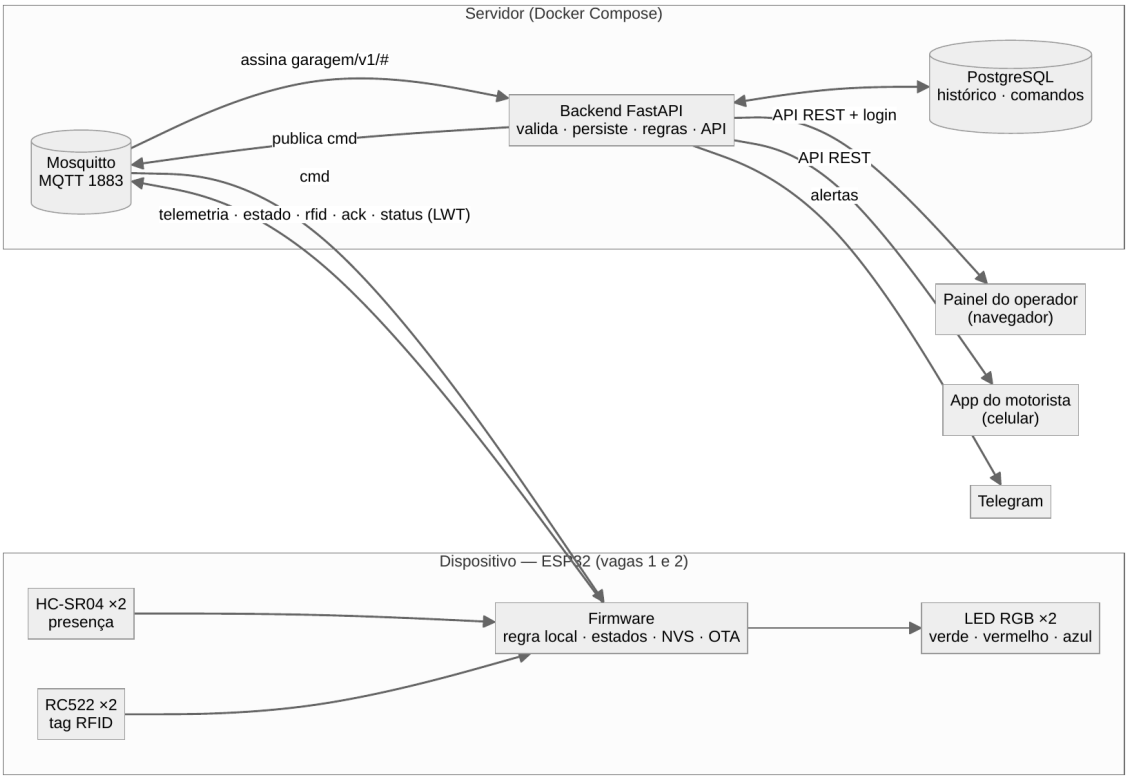
Req.	**Como o projeto vai atender**
R1 Hardware	ESP32 com **duas entradas de tipos diferentes** por vaga (HC-SR04 = distância/presença; RC522 = identificação RFID) em **dois pontos de medição** (vagas 1 e 2) e **atuador real**: LED RGB por vaga (verde/vermelho/azul) com resistor. Mapa de pinos (`docs/mapa-de-pinos.md`), BOM e esquema elétrico no repositório; maquete com fiação fixa.
R2 Firmware	Laço com `millis()` sem `delay()` longo (ultrassônico a cada 1 s, RFID a cada 100 ms); leituras validadas (faixa 2–400 cm, eco ausente como sentinela, leitor RC522 verificado a cada 5 s); **regra local sem rede**: LED segue o estado confirmado do sensor mesmo com Wi-Fi/broker fora; **estado seguro**: falha de sensor → vaga `erro`, LED vermelho piscando (nunca verde sem certeza); reconexão automática ao Wi-Fi e ao broker refazendo as assinaturas; **configuração persistente** em NVS (limiares e duração da reserva, alteráveis por comando `configurar`); `deviceId` derivado do MAC; timestamp via NTP ou campo `ts_sincronizado: false`; logs no Serial.
R3 Contrato MQTT	`CONTRATOS.md` com árvore de tópicos (telemetria, estado, rfid, cmd, ack, status/LWT), exemplo JSON por tópico com `v`, `deviceId`, `ts` e unidades, comandos aceitos e faixas, confirmação com `commandId` e resultado, catálogo de erros, QoS/retain justificados. v0 em 20/10; v1 congelado em 27/10 (seção 6).
R4 Backend	FastAPI + cliente MQTT assinando `garagem/v1/#`; validação com Pydantic contra o contrato (inválidas descartadas e gravadas em `mensagens_invalidas`); persistência no PostgreSQL; API REST: estado atual, histórico, alertas, envio de comando; único publicador de `cmd`.
R5 Banco de dados	PostgreSQL com tabelas `dispositivos`, `medicoes` (dispositivo, vaga, grandeza, valor, unidade, ts), `estados_vaga`, `leituras_rfid`, `comandos` (commandId, origem, ação, enviado_em, resultado, confirmado_em, motivo), `alertas`, `usuarios`. Migrações versionadas (Alembic).
R6 Interface	Painel web do operador: distância atual (cm) e estado de cada vaga, histórico (tabela e gráfico), último contato e online/offline do dispositivo, alertas ativos, botão de reserva e formulário de configuração (limiares, tempo de reserva). App do motorista: vagas livres e vaga destacada pela tag.
R7 Ciclo de comando	Comando `reservar` (e `cancelar_reserva`, `configurar`) com estados distintos gravados no banco e exibidos na interface: **enviado** (backend publicou) → **recebido** (ESP32 aceitou ou rejeitou, com motivo) → **executado** (LED ficou azul) → **confirmado** (estado `reservada` publicado e refletido na interface). Intertravamento: reservar vaga ocupada ou em erro é rejeitado com `VAGA_OCUPADA` / `VAGA_EM_ERRO`.
R8 Decisão remota	Regra no backend: reserva sem chegada do carro dentro do tempo configurado pelo operador (padrão 15 min, salvo no banco) → backend gera `cancelar_reserva` com o mesmo ciclo do R7 e registra alerta. Segunda regra: vaga em `erro` ou dispositivo `offline` $>$ 1 min → alerta (R15).

Req.	Como o projeto vai atender
R9 Segurança básica	Login com usuário/senha (hash bcrypt) e token JWT na interface e nas rotas de comando; só usuário autenticado envia comando. Credenciais de Wi-Fi e broker no ESP32 em `secrets.h` (ignorado pelo Git, já existe desde a N1); banco, broker e app em `.env` com `.env.example` versionado; Mosquitto com usuário/senha (fim do `allow_anonymous`).
R10 Falhas testadas	Plano com 4+ cenários, cada um com provocação, esperado, observado e evidência (log/vídeo/print): (1) queda do Wi-Fi → LED segue sensor local, reconecta e republica; (2) broker indisponível → idem, interface mostra offline via LWT; (3) falha de sensor (Echo desconectado) → `erro`, estado seguro e alerta; (4) comando inválido/duplicado → rejeitado com código, `commandId` repetido não reexecuta. Extra: RC522 desconectado → identificação para, vaga segue funcionando.
R11 Entrega executável	README com pré-requisitos, `.env.example`, `docker compose up` para broker/backend/banco, gravação do firmware (Arduino IDE/arduino-cli) e reaproveitamento declarado; release/tag `v1.0` da versão defendida; commits de todos os integrantes.
R12 Orquestração (Docker Compose)	`compose.yaml` com `mosquitto` (config + senhas), `postgres` (volume persistente), `backend` (FastAPI, também serve painel e app). Variáveis sensíveis via `.env`. Um único `docker compose up -d`.
R13 Múltiplos dispositivos	**Pendência – não atendido no desenho atual.** A equipe decidiu usar **um único ESP32** para as duas vagas, o que dá um só `deviceId`. O backend, o contrato (`deviceId` em todo tópico) e a interface já são desenhados para N dispositivos. Alternativa avaliada para atender: um ESP32 por vaga com o mesmo firmware (a placa reserva já está na BOM). Decisão pendente com o professor.
R14 Painel analítico	Gráficos na própria interface (Chart.js) com dados reais do banco: taxa de ocupação por hora do dia, tempo médio de permanência por vaga, uptime do dispositivo e tempo em `erro`.
R15 Notificação externa	Bot do Telegram acionado pelo backend quando um sensor entra em `erro`, o dispositivo fica `offline` $>$ 1 min ou uma reserva expira.
R16 Segurança avançada (TLS/perfis)	Perfis com permissões diferentes na API e na interface: **operador** (visualiza estado, histórico e alertas) e **administrador** (reserva vagas, altera configuração, gerencia tags). Verificado no backend em cada rota. TLS no MQTT avaliado como extra; se não implementado, justificativa registrada (rede local isolada).
R17 OTA	ArduinoOTA (ou `HTTPUpdate` baixando o binário do backend), com senha de OTA em `secrets.h`. Demonstração: gravar nova versão pela rede e conferir a versão publicada no status.
R18 Testes e CI	`pytest` no backend: validação de contrato (payloads válidos/inválidos), ciclo de comando e regra de expiração da reserva; GitHub Actions rodando os testes (e compilando o firmware com `arduino-cli`) a cada push/PR.

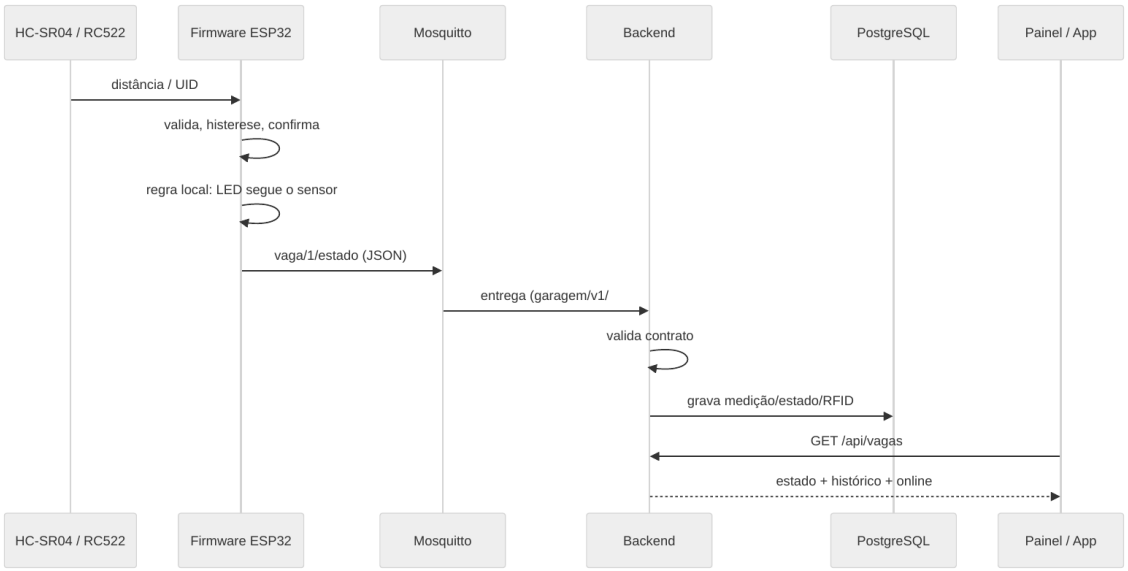
4. Arquitetura-alvo

Parte da entrega parcial de hoje (22h30).

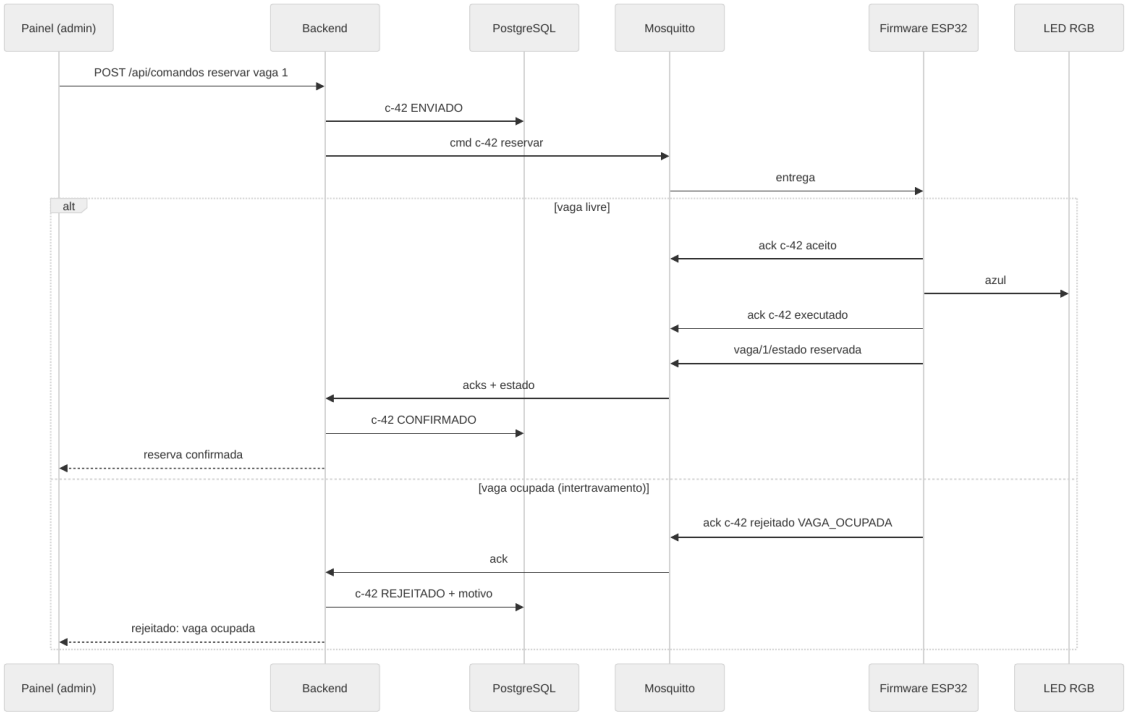
Desenhar (aqui ou em anexo) o diagrama de blocos e os dois fluxos da N3.



Fluxo do dado (sensor → firmware → MQTT → backend → banco → interface):



Fluxo do comando (interface → backend → MQTT → firmware → atuador → confirmação):



5. Hardware: desenho e dimensionamento

5.1 Verificação elétrica (calculada)

Sem bancada nesta etapa: os valores abaixo são **calculados a partir do datasheet** dos componentes escolhidos, não medidos. Anexar o esquema elétrico desenhado (à mão, Fritzing, KiCad ou equivalente) – não é preciso o circuito físico montado.

Verificação	**Valor calculado/esperado**	**Como será garantido**
Tensão no pino 3V3 e 5V/VIN usados	VIN = 5,0 V (USB); 3V3 = 3,3 V do regulador AMS1117 da placa (até 1 A). Carga no 3V3 ≈ ESP32 (pico ≈240 mA com Wi-Fi) + 2× RC522 (≈26 mA cada) ≈ 300 mA.	RC522 só no 3V3; HC-SR04 no VIN; fonte USB 5 V / 1 A (consumo total estimado ≈350 mA de pico).
Corrente estimada do(s) atuador(es)	LED vermelho: (3,3 − 2,0) / 220 Ω ≈ 5,9 mA. Verde/azul: (3,3 − 3,0) / 68 Ω ≈ 4,4 mA. Um canal aceso por vaga → ≤ 6 mA por GPIO, ≤ 12 mA no total.	Resistor em série em cada canal; abaixo do limite recomendado do GPIO (≈20 mA) e do absoluto (40 mA).
GND comum entre todas as fontes previstas	Uma única fonte (USB).	Trilha de GND única na protoboard ligando ESP32, HC-SR04, RC522 e LEDs.

Verificação	Valor calculado/esperado	Como será garantido
Maior tensão que chegará a um GPIO (qual sensor?)	HC-SR04 Echo = 5 V → divisor 1 kΩ / 2 kΩ: 5 × 2/3 = **3,33 V** nos GPIO 34/35. RC522 opera em 3,3 V.	Divisor em cada Echo (ou HC-SR04P de 3,3 V, sem divisor). Conferir com multímetro na montagem.
Proteção planejada para carga indutiva (se houver)	Não há carga indutiva (sem relé, motor ou bomba).	—
Estado seguro definido para o atuador no boot	LEDs **apagados** desde o `setup()` (pinos em `OUTPUT LOW`) até o primeiro estado confirmado do sensor; nenhum LED em pino de strapping.	Pinos dos LEDs (32, 33, 27, 13, 16, 17) sem função de boot; firmware inicializa antes de qualquer conexão.

Conferir no desenho:

☒ Nenhuma carga (motor, bomba, relé, cooler, fita de LED) alimentada diretamente por GPIO no esquema. *(Só LEDs indicadores com resistor, ≤ 6 mA.)*

☒ Carga indutiva com diodo de roda livre previsto (ou módulo que já o inclua — indicar qual). *(Não se aplica: sem carga indutiva.)*

☒ Nenhum sinal de 5 V ligado direto a um GPIO no desenho (divisor ou conversor de nível quando necessário). *(Divisor no Echo do HC-SR04.)*

☒ Nenhum uso de GPIO 6–11; nenhum atuador em pino de *strapping* (0, 2, 5, 12, 15) sem justificativa. *(GPIO 5 usado só como SS do RC522 — SS padrão do SPI, com pull-up interno que mantém o leitor desselecionado e o nível exigido no boot.)*

☒ Sensores analógicos planejados no ADC1 (GPIO 32–39), já que o ADC2 não lê com Wi-Fi ativo. *(Não se aplica: HC-SR04 e RC522 são digitais.)*

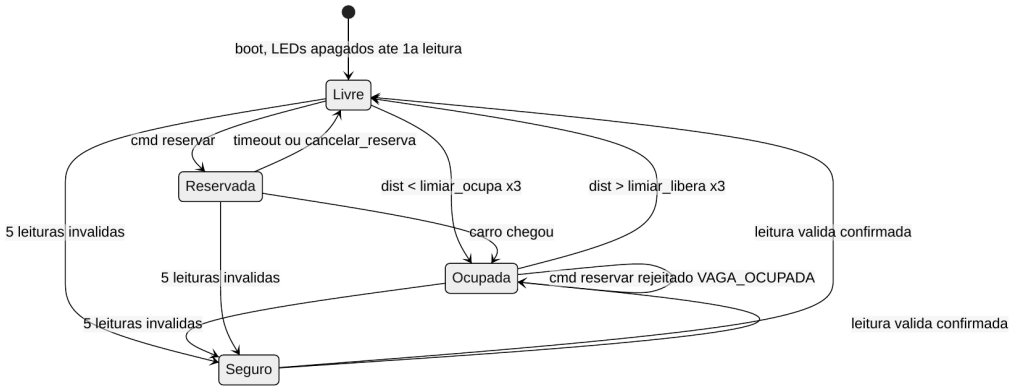
Esquema elétrico: pendente (Fritzing), a versionar em `docs/`.

5.2 BOM e mapa de pinos

BOM – o que já tem	1× ESP32 DevKit 30 pinos (ESP32-WROOM-32, USB-C, CP2102). *(Equipe completar.)*
BOM – o que falta comprar/providenciar	1× ESP32 reserva; 2× HC-SR04; 2× RC522 + 4–6 tags 13,56 MHz (MIFARE 1K); 2× LED RGB catodo comum; resistores 2× 220 Ω, 4× 68–100 Ω, 2× 1 kΩ, 2× 2 kΩ; 2 protoboards 830 furos; jumpers; cabo USB-C de dados; fonte USB 5 V / 1 A; roteador Wi-Fi 2,4 GHz; servidor (PC/notebook/Raspberry Pi); maquete (base, suportes, carrinhos).
Caminho do mapa de pinos no repositório	`docs/mapa-de-pinos.md`

5.3 Fluxograma do firmware

Desenhar os estados do firmware (pelo menos automático, manual e seguro), indicando em cada transição o evento que a provoca, o timeout do modo manual e a condição de inter- travamento. É um fluxograma de design: a implementação real no ESP32 pode ajustar detalhes.



Estado	Modo	LED	Saída
Livre / Ocupada	**Automático** (regra local, funciona sem rede)	verde / vermelho	sensor confirma mudança (3 leituras)
Reservada	**Manual** (por comando)	azul	**timeout** da reserva (`duracao_min`, padrão 15 min, em NVS — conta mesmo sem rede), `cancelar_reserva` ou carro detectado
Seguro	**Seguro** (falha de sensor, publica `erro`)	vermelho piscando, nunca verde	leitura válida confirmada

Intertravamento: `reservar` só é aceito com a vaga **Livre**; em Ocupada → `VAGA_OCUPADA`, em Seguro → `VAGA_EM_ERRO`.

Paralelo aos estados, o laço principal (sem `delay()` longo): verifica Wi-Fi/MQTT e reconecta refazendo a assinatura de `cmd` → lê ultrassônico (1 s) → lê RFID (100 ms) → atualiza LED → publica pendentes → processa comandos (aceita/rejeita, executa, confirma) → telemetria periódica (10 s). Detalhes em `docs/fluxograma-firmware.md`.

6. Contrato v0

Na entrega parcial de hoje, ainda não precisa estar congelado. Na entrega completa (final da Aula 13), já deve estar congelado como v1. Escrever a árvore de tópicos pretendida e um exemplo de payload.

Congelado como v1 em 27/10/2026. Contrato completo, com exemplo de payload por tópico, comandos, faixas e códigos de erro, em `CONTRATOS.md`.

Árvore de tópicos	`garagem/v1/{id}/telemetria` QoS 0, sem retain `garagem/v1/{id}/vaga/{n}/estado` QoS 1, retain `garagem/v1/{id}/rfid` QoS 1, sem retain `garagem/v1/{id}/cmd` QoS 1, sem retain `garagem/v1/{id}/ack` QoS 1, sem retain `garagem/v1/{id}/status (LWT)` QoS 1, retain `{id}` = `deviceId` (ex.: `esp32-garagem-240ac4000110`). Estado: `livre` / `ocupada` / `reservada` / `erro`. Comandos: `reservar`, `cancelar_reserva`, `configurar`. Ack: `aceito` / `rejeitado` / `executado` + `commandId`.

Exemplo de payload (telemetria ou estado)	`// garagem/v1/esp32-garagem-240ac4000110/vaga/1/estado`
	`{ "v": 1, "deviceId": "esp32-garagem-240ac4000110", "ts": "2026-10-27T21:30:05Z", "vaga": 1, "estado": "ocupada", "distancia": { "valor": 12.4, "unidade": "cm"} }`
	`// garagem/v1/esp32-garagem-240ac4000110/ack`
	`{ "v": 1, "deviceId": "esp32-garagem-240ac4000110", "ts": "2026-10-27T21:31:01Z", "commandId": "c-0042", "resultado": "rejeitado", "erro": "VAGA_OCUPADA"}`

7. Tecnologias pretendidas

Escolha livre de linguagem, framework, banco e interface (ver seção 4.1 de `requisitos_projeto_N3.pdf`): vedadas plataformas prontas e fechadas em que backend, regras, banco ou interface não possam ser inspecionados no repositório (ex.: Blynk, Arduino IoT Cloud). Em caso de dúvida, perguntar antes de escolher.

Camada	**Tecnologia escolhida**	**Justificativa**
Backend	Python 3.12 + FastAPI + aiomqtt	Sugestão da disciplina; validação de contrato com Pydantic; documentação automática da API (OpenAPI); `pytest` simples para R18.
Banco de dados	PostgreSQL 16 + SQLAlchemy/Alembic	Relacional recomendado; consultável ao vivo (psql); migrações versionadas para R5.
Interface	Web (HTML/CSS/JS) servida pelo backend + Chart.js; app do motorista responsivo (PWA)	Evolui o app da N1 sem build; roda no navegador do celular sem instalação; gráficos do R14 sem ferramenta externa.
Broker MQTT	Eclipse Mosquitto 2	Já usado na N1; leve, aberto, suporta usuário/senha, ACL, WebSocket e TLS.
Orquestração (Docker Compose)	Docker Compose: `mosquitto`, `postgres`, `backend`	Sobe os três serviços com `docker compose up -d`; segredos em `.env`.

8. Cronograma, riscos e divisão de responsabilidades

8.1 Cronograma até a N3

Data	Marco	O que a equipe entrega
27/10	CP12 completo (fim da Aula 13) + contrato congelado v1	PDF completo; `CONTRATOS.md` v1; esquema elétrico; fluxograma do firmware; modelo de dados; esboço das telas; plano de testes dos 4 cenários; repositório atualizado com commit de todos.
03/11	N2	T2 (este documento + repositório) e prova individual P2.
10/11 – 24/11	Unidade 3	**10/11:** hardware montado na maquete, firmware com contrato v1, regra local, NVS (R1–R3); `compose.yaml` com broker + banco + backend esqueleto (R12). **17/11:** backend validando e persistindo, API, painel com login e ciclo de comando (R4–R7, R9). **24/11:** regra remota, Telegram, gráficos, perfis, OTA, testes + CI, falhas testadas com evidência (R8, R10, R14–R18).
01/12	N3	Release `v1.0` no GitHub; README executável (R11); defesa do produto funcionando.

8.2 Riscos

Risco	Mitigação
R13 não atendido com um único ESP32.	Backend e contrato já suportam N dispositivos; decisão pendente com o professor; placa reserva na BOM permite dividir uma vaga por ESP32 sem mudar o firmware.
Atraso na compra de RC522/tags/LED RGB.	RFID é complementar — o sistema funciona só com ultrassônico; tags podem ser publicadas manualmente (`mosquitto_pub`) nos testes.
Dois RC522 no mesmo SPI não se entenderem.	SS separados, todos em nível alto antes do `SPI.begin`; verificação periódica com reinicialização; testar cedo (10/11).
Rede da faculdade bloqueia ESP32/celulares (WPA2-Enterprise, isolamento).	Roteador 2,4 GHz próprio para a demonstração.
Calibração do ultrassônico na maquete.	Limiares configuráveis pela interface (NVS) — ajuste sem regravar.
Escopo grande (R12–R18) para 3 aulas.	Priorizar R1–R11 em 10/11 e 17/11; R12 desde o início (compose); R14–R18 em 24/11; divisão por camada.
ESP32 único é ponto único de falha.	LWT + alerta; estado seguro; regra local sem rede; placa reserva.

8.3 Divisão de responsabilidades

Integrante	**Responsabilidade principal até a entrega completa (Aula 13)**
Gustavo Vinicius Taques	Hardware: esquema elétrico (Fritzing), verificação elétrica, BOM
Luis Fernando Pereira	Firmware: fluxograma de estados, mapa de pinos, contrato v1 (`CONTRATOS.md`)
João Pedro Angelico	Backend e banco: modelo de dados, endpoints da API, regra remota, `compose.yaml` planejado
Vynicyus Candido	Interface: esboço das telas (painel e app do motorista), plano de testes de falha, repositório/README

9. Repositório (GitHub) – 1,0 ponto fixo (25% do T2)

O repositório é a segunda entrega de hoje, junto com este PDF (ver Orientações). Precisa existir com um `README.md` simples – nome do projeto, problema e usuário, estado atual (herdado da N1) e para onde o upgrade está indo – desde às 22h30. Não é o README de instalação completo: isso só é exigido para a N3 (R11).

A partir de 20/10/2026, a completude do repositório tem nota própria e fixa dentro do T2: 1,0 ponto dos 4,0, avaliado na entrega completa (final da Aula 13). O princípio é simples: o PDF é a proposta, o repositório é a prova – o que está escrito aqui precisa existir, de fato, versionado no GitHub.

Critério de avaliação:

- **Completo (1,0 ponto)** – todos os itens presentes: repositório **público** (acessível sem convite); `README.md` completo (projeto, problema, usuário, estado atual, direção do upgrade); **diagramas e gráficos** versionados no repositório (arquitetura-alvo, fluxograma do firmware, esquema elétrico – não apenas texto); conteúdo do repositório **correspondendo exatamente** ao que foi entregue em PDF (mesmo commit, sem divergência); **commit de cada integrante** da equipe.
- **Parcial (0,5 ponto)** – falta um item secundário: por exemplo, só texto sem diagramas versionados, ou commits concentrados em 1–2 integrantes.
- **Insuficiente (0 ponto)** – repositório não existe, não é público, ou diverge do PDF entregue.

Link do repositório	`https://github.com/VynicyusCandido/iotHorangoTango`
Hash do commit entregue	tag `cp12-entrega` – commit que contém este PDF e o `docs/cp12/CP12.tex` que o gerou

Integrante	**Usuário GitHub (com commit próprio no repositório)**
Gustavo Vinicius Taques	*(pendente: sem commit no repositório)*

Luis Fernando Pereira	Luis-Fernando-Pereira
João Pedro Angelico	joaopedroangelico
Vynicyus Candido	VynicyusCandido

Checklist antes do envio

☒ Seções 1 a 4 enviadas até 22h30 de hoje (mesmo que o resto ainda esteja em branco).

☒ Repositório no GitHub criado/atualizado até 22h30, com `README.md` simples apontando para onde o projeto está indo. **Repositório público** – o ponto do GitHub (seção 9) exige acesso sem convite.

☒ Tabela de verificação elétrica preenchida com valores **calculados**, não medidos – sem bancada nesta etapa.

☐ Esquema elétrico desenhado anexado (não é necessário circuito físico montado).

☒ Fluxograma do firmware desenhado.

☒ Conferência R1–R18 sem linha em branco (pendências declaradas, não omitidas).

☒ Cronograma até a N3 preenchido.

☒ Tecnologias pretendidas justificadas; nenhuma plataforma fechada da seção 4.1.

☐ Repositório com commit de cada integrante listado.

☐ Todos os integrantes revisaram o documento completo antes do envio final.

Entrega

Duas etapas, cada uma com duas coisas – **PDF no Teams** e **repositório no GitHub**:

1. **Até 22h30 de hoje (20/10):** PDF com pelo menos as seções 1 e 3 (nome sugerido: `CP12_EQUIPE-X_PARCIAL.pdf`) e repositório criado/atualizado com `README.md` simples (projeto, problema, usuário, estado atual, direção do upgrade).
2. **Até o final da Aula 13 (27/10/2026, 22h30):** PDF completo, com todas as seções (nome sugerido: `CP12_EQUIPE-X_PROJETO_v1.0.pdf`) e repositório atualizado de novo, com o contrato já congelado como v1.